

SOFTWARE REQUIREMENT SPECIFICATION

(Specialization Project - C1 Evaluation)

Agentic RAG-Based Citizen Service Advisory Chatbot

GitHub Repository: github.com/abhi6744/Agentic-RAG-Based-Citizen-Service-Advisory-Chatbot

Submitted by Abhinav G

Roll No: 25225040

Under the guidance of

Dr. Ramesh Chandra Poonia

Dr. Shilpa Shrivastava



CHRIST

(DEEMED TO BE UNIVERSITY)

BANGALORE | DELHI NCR | PUNE

School of Sciences

2026-2027

1. INTRODUCTION

1.1 Purpose

This document specifies the functional and non-functional requirements for the Citation-Grounded Multilingual RAG Assistant. It serves as the formal agreement between the project idea, the expected system behavior, and the evaluation criteria. In practical terms, it explains what the system must do, how it should behave, who will use it, and what limits must be respected during implementation. The document is written for the project mentor, evaluator, and the developer so that the scope remains clear throughout the project lifecycle.

1.2 Scope

The system is a citizen-facing and operator-facing information assistant for Akshaya service centers in Kerala. It retrieves answers from official documents such as government service pages, FAQs, PDFs, circulars, and service instructions, then generates grounded responses with citations. The assistant focuses on service eligibility, required documents, fees, office/process steps, and general guidance. It does not perform form submission, payment processing, legal judgments, or medical decision-making; its role is strictly informational and supportive. This makes the project useful in a real deployment setting while keeping the scope manageable for a student implementation.

1.3 Definitions and Abbreviations

Term	Definition
RAG	Retrieval-Augmented Generation
LLM	Large Language Model
Embedding	Numeric vector representation of text used for similarity search
Chunk	A small text segment extracted from a larger source document
Confidence Score	A numeric measure of how strongly retrieved evidence supports an answer
Citation	A reference linking a generated answer to its exact source document/section

2. OVERALL DESCRIPTION

2.1 Product Perspective

The system is a standalone information-assistance layer that sits above the official document repository. Citizens or operators submit a question, the system searches the curated knowledge base, and the LLM turns the retrieved evidence into a readable answer. The system can be deployed on a small web server, a local machine, or a cloud-hosted environment, and it does not require modification of existing government systems. In simple words, it acts as a smart search-and-answer layer on top of official service information.

2.2 Product Functions (Summary)

- Accept multilingual natural-language queries.
- Detect the query language and route it properly.
- Retrieve relevant document chunks from a vector store.
- Generate an answer only from the retrieved evidence.
- Attach source citations to the final answer.
- Show eligibility, required documents, fees, and steps.
- Warn the user when evidence is weak or incomplete.
- Allow browsing by service category.
- Let administrators update the knowledge base.
- Log queries and responses for testing and improvement.

2.3 User Characteristics

The main user groups are citizens and Akshaya operators. Citizens may have low digital literacy, so the interface must be simple, clear, and language-friendly. Operators need faster access to precise information so they can serve more people with fewer mistakes. Administrators will manage documents and system updates, so they require slightly more technical comfort than ordinary users.

2.4 Constraints

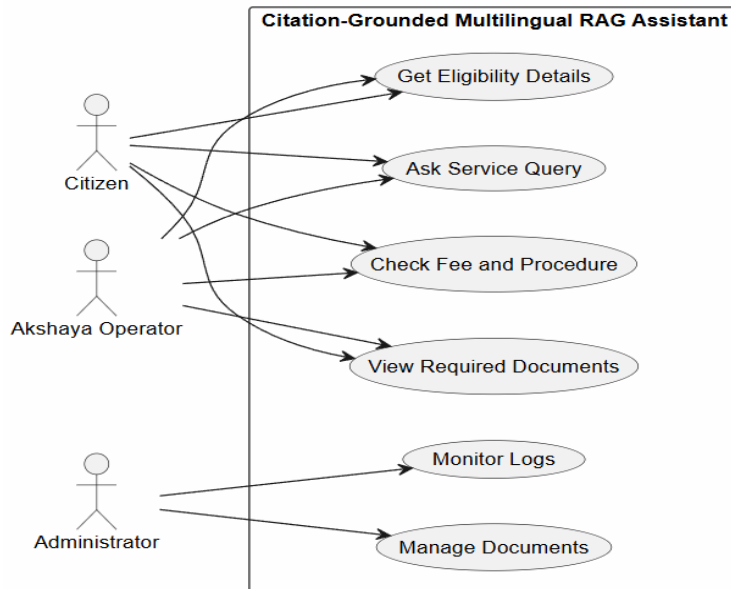
- The project must be completed within one academic trimester.
- The system must use only verified, official, or government-backed information.
- The response time should remain low enough for practical use.
- The system should not rely on private data or hidden proprietary sources.
- The first version should cover only a limited set of high-demand services.

2.5 Assumptions and Dependencies

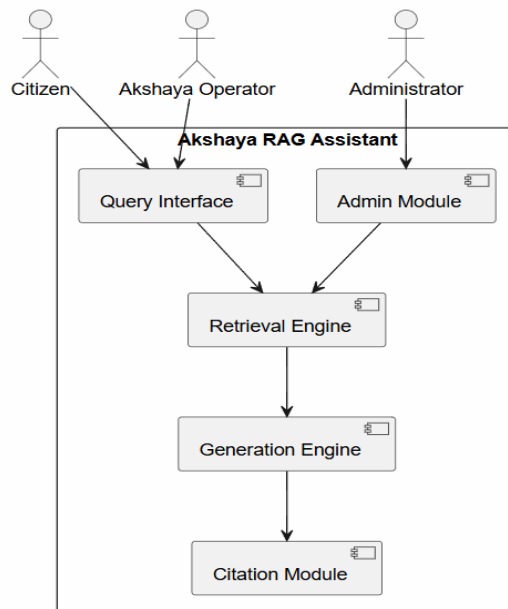
- Official information is available in digital form for ingestion.
- The language model or API used for generation remains accessible during development.
- Internet connectivity is available where needed for deployment or testing.
- Service rules may change over time, so the knowledge base may need updates.

3. DIAGRAM SPACE FOR SRS

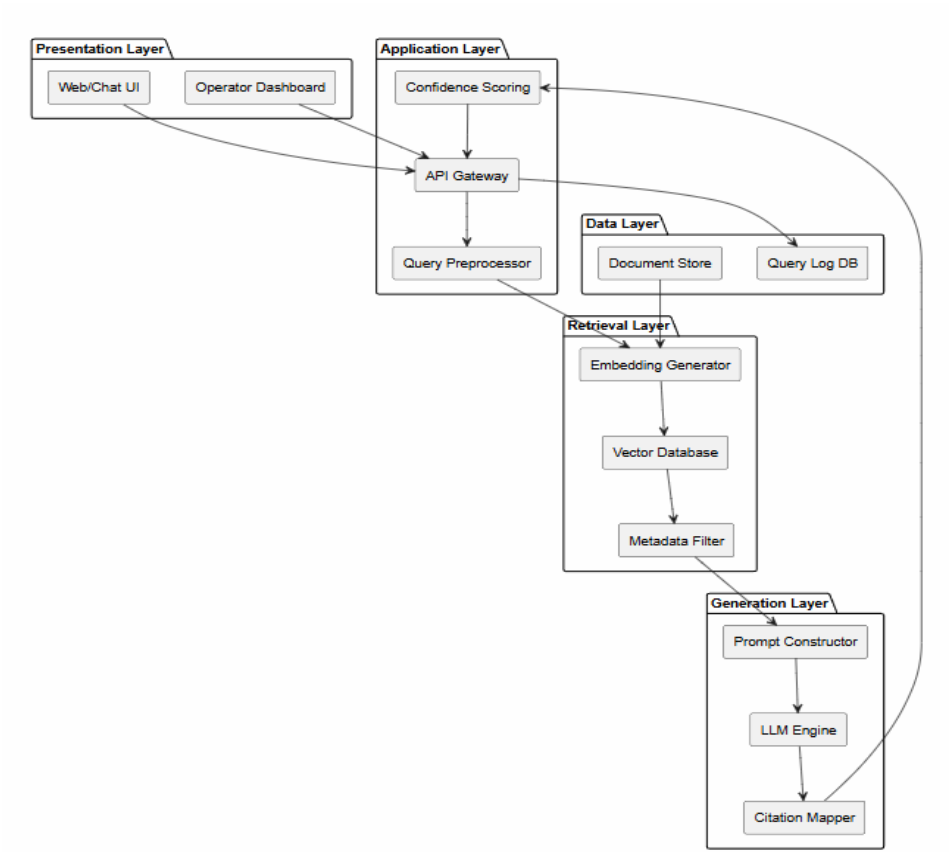
3.1 Use Case Diagram



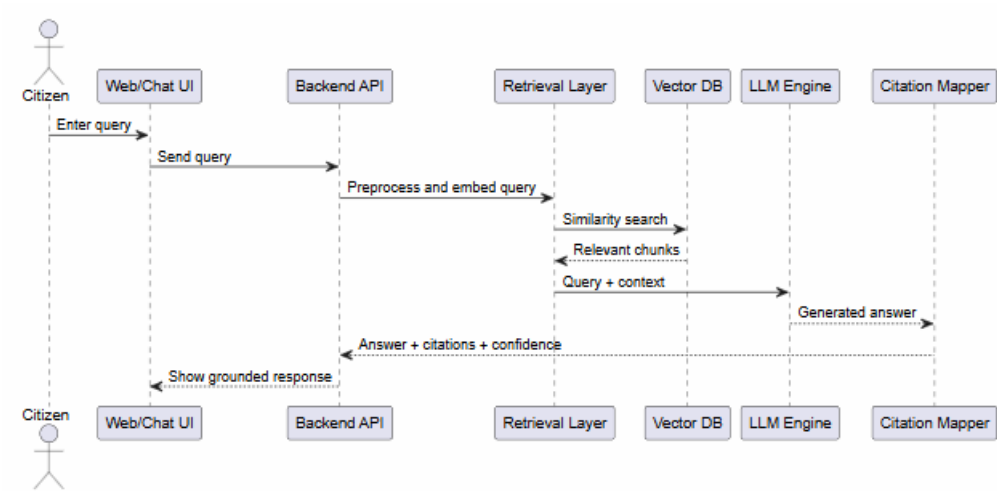
3.2 Context Diagram



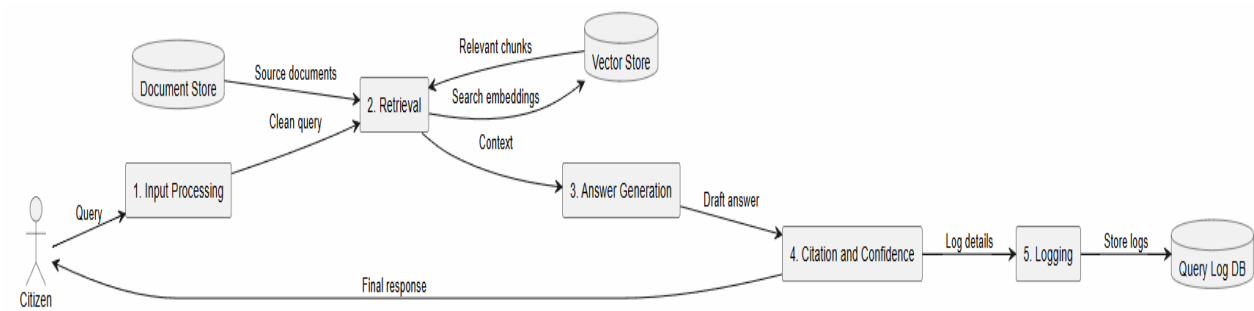
3.3 Architecture Diagram



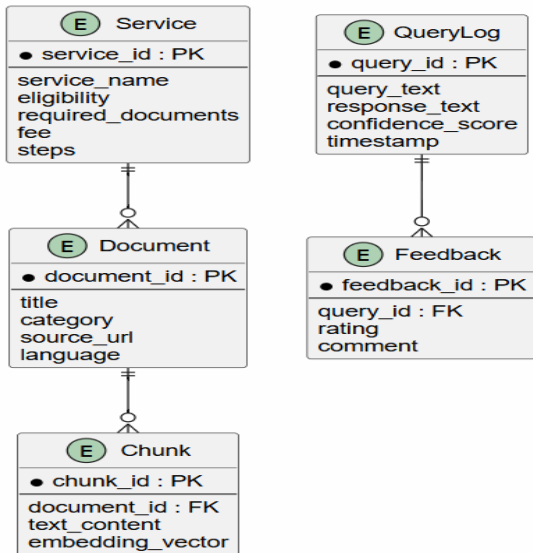
3.4 Sequence Diagram



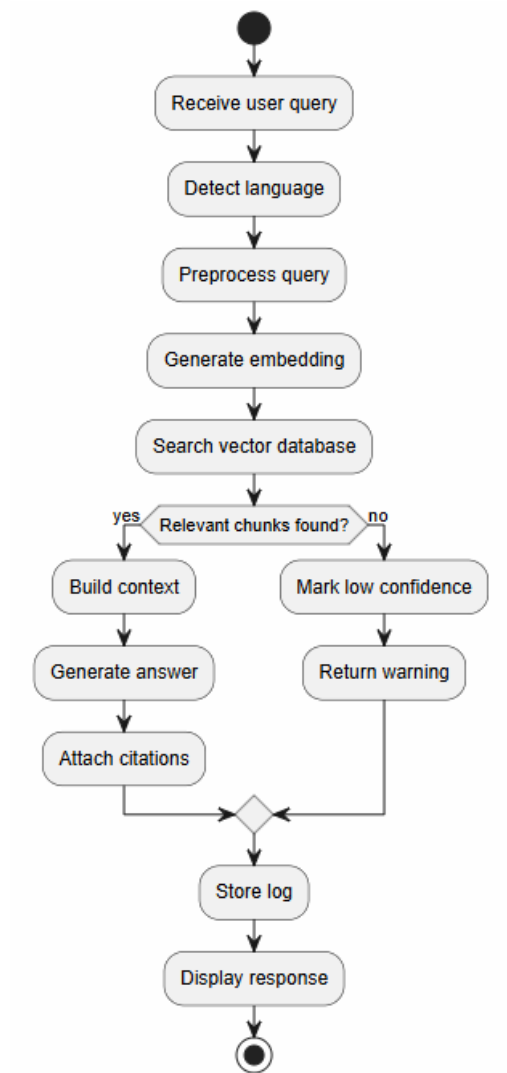
3.5 Data Flow Diagram



3.6 ER Diagram



3.8 Flowchart



4. FUNCTIONAL REQUIREMENTS

4.1 Query Handling

ID	Requirement
FR-1	The system shall accept user queries in English and Malayalam text.
FR-2	The system shall detect the query language and route it through the appropriate embedding pipeline.
FR-3	The system shall normalize and preprocess queries before embedding generation.

The query handling module is important because the users of Akshaya centers may ask questions in different forms, spellings, or mixed languages. The preprocessing step removes unnecessary noise and makes the query easier to match with official documents. Language detection helps the system choose the correct processing path and improves retrieval accuracy. Without this step, the system may search in the wrong language or miss relevant documents entirely.

4.2 Retrieval

ID	Requirement
FR-4	The system shall generate an embedding vector for every incoming query.
FR-5	The system shall perform similarity search against the vector database to retrieve the top-k relevant chunks.
FR-6	The system shall support metadata filtering by service category and language.

Retrieval is the core of the system because it ensures the answer is based on trusted source material instead of the LLM's memory. The document chunks should be small enough to search efficiently but large enough to preserve meaning. Metadata filtering helps the system avoid mixing unrelated services or wrong-language documents. This improves both relevance and trustworthiness.

4.3 Generation and Citation

ID	Requirement
FR-7	The system shall construct a prompt combining the user query and retrieved context.
FR-8	The system shall generate an answer using the LLM strictly from retrieved context.
FR-9	The system shall map every generated claim to its source document and display the citation.
FR-10	The system shall compute a confidence score based on retrieval similarity.
FR-11	The system shall display a warning when the confidence score falls below a configurable threshold.

This module is what turns the project into a safe and useful RAG system rather than a generic chatbot. The answer should be readable and helpful, but it must remain constrained by the retrieved sources. Citation display is important because it allows the mentor, user, or evaluator to verify where the answer came from. Confidence signaling is equally important because the system should admit uncertainty when evidence is weak.

4.4 Service Information Display

ID	Requirement
FR-12	The system shall display eligibility conditions for the queried service.
FR-13	The system shall display a required-document checklist for the queried service.
FR-14	The system shall display applicable fees and step-by-step procedures.
FR-15	The system shall allow browsing of services by category without requiring a typed query.

This section ensures the output is practical for real citizens. Most service-related confusion happens because people do not know whether they qualify, what to bring, or which office step to follow. By presenting these details in one response, the system saves time for both citizens and operators. Browsing by category also supports users who do not know the exact service name.

4.5 Administration

ID	Requirement
FR-16	The system shall allow an administrator to upload, update, or remove source documents.
FR-17	The system shall re-index the knowledge base automatically after a document update.
FR-18	The system shall log every query, response, and confidence score for evaluation.
FR-19	The system shall allow users to submit feedback on answer usefulness.

Administration is needed because the source documents will change over time. A good system must support updates without rebuilding everything from scratch. Logging helps in evaluation, debugging, and performance comparison, while feedback allows future improvement. This is also useful for a viva discussion because it shows that the project is not just a demo but a maintainable system.

5. NON-FUNCTIONAL REQUIREMENTS

Category	Requirement
Performance	The system shall return a response within 5 seconds under normal load.
Accuracy	The system shall achieve at least 85% citation accuracy during evaluation testing.
Usability	The interface shall be usable by citizens with minimal digital literacy without training.
Category	Requirement
Scalability	The architecture shall support addition of new services and documents without major redesign.
Security	The system shall not store personally identifiable information beyond anonymized query logs.
Reliability	The system shall handle invalid or unclear queries gracefully without crashing.
Maintainability	The knowledge base shall be updatable independently of core application logic.
Availability	The system shall be available during standard Akshaya center working hours with minimal downtime.

These requirements are important because the project is meant to operate in a public-service environment. The system must be fast enough to be useful at a service center counter, accurate enough to be trusted, and simple enough for citizens to understand. Reliability and maintainability matter because the underlying documents and service rules will not remain fixed forever. Security is also necessary because even a guidance system should respect user privacy.

6. EXTERNAL INTERFACE REQUIREMENTS

6.1 Software Interfaces

- LLM Provider API — accessed only through the generation module.
- Vector Database (FAISS/Chroma) — accessed through the retrieval module.
- SQLite/PostgreSQL — persistence for metadata, query logs, and feedback.
- Web frontend or dashboard — used by citizens, operators, and administrators.

6.2 Hardware Interfaces

None beyond a standard computer or mobile device with internet access.

6.3 User Interfaces

A simple web-based chat interface for citizens/operators, a service-browsing panel, and an administrator dashboard for document management. The interface should avoid technical jargon and should show clear answer sections such as eligibility, required documents, fees, and source citations.

7. USE CASE SUMMARY

Use Case	Primary Actor	Description
Ask Service Query	Citizen/Operator	Submits a natural-language query and receives a grounded, cited answer.
Browse Services	Citizen/Operator	Browses available services by category without typing a query.
Manage Documents	Administrator	Uploads, updates, or removes source documents from the knowledge base.
Submit Feedback	Citizen/Operator	Rates whether a given answer was helpful.
Monitor Query Logs	Administrator	Reviews logged queries and confidence scores for system improvement.

8. SECURITY AND PRIVACY

The system shall not expose raw API keys to the user interface. It shall avoid collecting personal user data unless necessary for query logging, and such logs should be anonymized. Access to the administrator dashboard should be restricted to authorized users only. Any uploaded document should be checked before indexing to avoid corrupt or irrelevant content entering the system.

9. VALIDATION AND TESTING

9.1 Validation Criteria

- Retrieval relevance should be checked against a curated test set.
- Citation accuracy should be verified manually for sample responses.
- Response latency should be measured under normal load.
- Multilingual query handling should be tested with English and Malayalam inputs.
- Low-confidence warnings should be triggered for weak or ambiguous queries.

9.2 Test Cases

- Query about service eligibility.
- Query about required documents.
- Query about fee and process.
- Query with incomplete information.
- Query with low-confidence retrieval match.

Testing is necessary to prove that the system works beyond the conceptual level. A strong SRS should make testing measurable. For example, accuracy is not just a claim; it should be checked with sample queries and source comparisons. Likewise, latency should be measured because a helpful system must feel responsive in a real service center setting.